SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
ProgramName: B. Tech		**Assignment Type: Lab**	**AcademicYear:** 2025-2026
CourseCoordinatorName		Venkataramana Veeramsetty	
Instructor(s)Name		Dr. V. Venkataramana (Co-ordinator) Dr. T. Sampath Kumar Dr. Pramoda Patro Dr. Brij Kishor Tiwari Dr.J.Ravichander Dr. Mohammand Ali Shaik Dr. Anirodh Kumar Mr. S.Naresh Kumar Dr. RAJESH VELPULA Mr. Kundhan Kumar Ms. Ch.Rajitha Mr. M Prakash Mr. B.Raju Intern 1 (Dharma teja) Intern 2 (Sai Prasad) Intern 3 (Sowmya) NS_2 (Mounika)	
CourseCode	24CS002PC215	**CourseTitle**	AI Assisted Coding
Year/Sem	II/I	**Regulation**	R24
Date and Day of Assignment	Week1 - Thursday	**Time(s)**	
Duration	2 Hours	**Applicableto Batches**	24CSBTB01 To 24CSBTB39
AssignmentNumber: **2.4**(Present assignment number)/**24**(Total number of assignments)			

Q.No.	Question	*ExpectedTime to complete*
1	Lab 2: Exploring Additional AI Coding Tools – Gemini (Colab) and Cursor AI	Week1 - Thursday

Lab Objectives:

- To explore and evaluate the functionality of Google Gemini for AI-assisted coding within Google Colab.
- To understand and use Cursor AI for code generation, explanation, and refactoring.
- To compare outputs and usability between Gemini, GitHub Copilot, and Cursor AI.
- To perform code optimization and documentation using AI tools.

Lab Outcomes (LOs):
After completing this lab, students will be able to:

- Generate Python code using Google Gemini in Google Colab.
- Analyze the effectiveness of code explanations and suggestions by Gemini.
- Set up and use Cursor AI for AI-powered coding assistance.
- Evaluate and refactor code using Cursor AI features.
- Compare AI tool behavior and code quality across different platforms.

Task Description #1
• Open Google Colab and use Google Gemini to generate Python code that performs sorting of a list using both the bubble sort algorithm and Python's built-in sort() function. Compare the two implementations.

Expected Output #1
• Two sorting implementations: Bubble sort (manual logic) and Built-in sort()

PROMPT:

Generate Python program that performs sorting of a list using both the bubble sort algorithm and Python's built-in sort() function. Compare the two implementations

CODE:

```python
import time
import random
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr
data = [random.randint(0, 1000) for _ in range(1000)]
start_time = time.time()
bubble_sorted_data = bubble_sort(data.copy())
end_time = time.time()
bubble_sort_time = end_time - start_time
print(f"Bubble Sort Time: {bubble_sort_time:.6f} seconds")
start_time = time.time()
python_sorted_data = sorted(data.copy())
end_time = time.time()
python_sort_time = end_time - start_time
print(f"Python's Built-in Sort Time: {python_sort_time:.6f} seconds")
print(f"Are the results the same? {bubble_sorted_data == python_sorted_data}")
```

```
Bubble Sort Time: 0.048506 seconds
Python's Built-in Sort Time: 0.000206 seconds
Are the results the same? True
```

OBSERVATION:

The bubble sort algorithm and compares its performance with Python's built-in sorted() function on a randomly generated list of 1000 integers. As expected, Python's built-in sort is significantly faster than the bubble sort algorithm for this input size. The output also confirms that both sorting methods produce the same sorted list.

Task Description #2

• In Colab, use Google Gemini to generate a Python function that takes a string and returns:
The number of vowels, The number of consonants, The number of digits in the string

Expected Output #2-

• Complete function that Iterates through characters of a string and Counts vowels, consonants, and digits

PROMPT:

Generate a Python function that takes a string and returns:
The number of vowels, The number of consonants, The number of digits in the string

CODE:

```python
def count_chars(input_string):
    vowels = "aeiouAEIOU"
    vowel_count = 0
    consonant_count = 0
    digit_count = 0

    for char in input_string:
        if char.isalpha():
            if char in vowels:
                vowel_count += 1
            else:
                consonant_count += 1
        elif char.isdigit():
            digit_count += 1

    return vowel_count, consonant_count, digit_count
test_string = "Hello World 123!"
vowels, consonants, digits = count_chars(test_string)
print(f"Input string: '{test_string}'")
print(f"Number of vowels: {vowels}")
print(f"Number of consonants: {consonants}")
print(f"Number of digits: {digits}")
```

```
Input string: 'Hello World 123!'
Number of vowels: 3
Number of consonants: 7
Number of digits: 3
```

OBSERVATION:

The code effectively defines a function count_chars that iterates through a given string and accurately categorizes each character as a vowel, consonant, or digit, incrementing the respective counters. The example usage with the string "Hello World 123!" demonstrates the function's correct behavior, showing the expected counts for vowels, consonants, and digits.

Task Description #3

• Install and set up Cursor AI. Use it to generate a Python program that performs file handling:
Create a text file
Write sample text
Read and display the content

Expected Output #3

• Functional code that creates a .txt file, writes content to it, and reads it back.
• Screenshot of Cursor AI interface showing: Prompt used, Generated code, Output of file operations

PROMPT:

Generate a Python program that performs file handling:
Create a text file
Write sample text
Read and display the content

CODE:

```python
file_name = "sample.txt"
sample_text = "This is a sample line.\nThis is another line."
try:
    with open(file_name, "w") as file:
        file.write(sample_text)
    print(f"Successfully created '{file_name}' and wrote sample text.")
except IOError as e:
    print(f"Error writing to file: {e}")
try:
    with open(file_name, "r") as file:
        file_content = file.read()
    print(f"\nContent of '{file_name}':")
    print(file_content)
except IOError as e:
    print(f"Error reading file: {e}")
```

```
Successfully created 'sample.txt' and wrote sample text.

Content of 'sample.txt':
This is a sample line.
This is another line.
```

OBSERVATION:

The code successfully demonstrates basic file handling in Python. It first creates a text file named "sample.txt" and writes the sample_text into it using the with open(...) statement, which ensures the file is properly closed even if errors occur. The output confirms the successful creation and writing. Then, the code opens the same file in read mode ("r") and reads its entire content, which is then printed to the console. The output shows the content of the file as expected. Error handling for IOError is also included in both the writing and reading parts.

This is a good basic example of how to create, write to, and read from a file in Python.

Task Description #4

• Ask Google Gemini to generate a Python program that implements a simple calculator using functions (add, subtract, multiply, divide). Then, ask Gemini to explain how the code works.

Expected Output #4

• Complete calculator code with user input and operation selection.

• Line-by-line explanation or markdown-style explanation provided by Gemini.

• Screenshot of both the code and explanation in Colab.

PROMPT:

Generate a Python program that implements a simple calculator using functions (add, subtract, multiply, divide). Then, ask Gemini to explain how the code works

CODE:

```python
def add(x, y):
  return x + y
def subtract(x, y):
  return x - y
def multiply(x, y):
  return x * y
def divide(x, y):
  if y == 0:
    return "Error! Division by zero."
  else:
    return x / y
num1 = 10
num2 = 5
print(f"{num1} + {num2} = {add(num1, num2)}")
print(f"{num1} - {num2} = {subtract(num1, num2)}")
print(f"{num1} * {num2} = {multiply(num1, num2)}")
print(f"{num1} / {num2} = {divide(num1, num2)}")
num3 = 10
num4 = 0
print(f"{num3} / {num4} = {divide(num3, num4)}")
```

```
10 + 5 = 15
10 - 5 = 5
10 * 5 = 50
10 / 5 = 2.0
10 / 0 = Error! Division by zero.
```

OBSERVATION:

The divide function includes error handling to prevent division by zero, returning a specific message in that case. The example usage demonstrates how to call these functions with different numbers and prints the results, including the division by zero scenario.

Task Description #5

• Use Cursor AI to create a Python program that checks if a given year is a leap year or not. Try different prompt styles and see how Cursor modifies its code suggestions.

Expected Output #5

• A functional program to check leap year with sample input/output
• At least two versions of the code (from different prompts)
• A short comparison of which version is better and why

PROMPT:

Create a Python program that checks if a given year is a leap year or not. Try different prompt styles and see how Cursor modifies its code suggestions

CODE:

```python
def is_leap_year(year):
  if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):
    return True
  else:
    return False
year1 = 2024
year2 = 1900
year3 = 2000
year4 = 2023
print(f"{year1} is a leap year: {is_leap_year(year1)}")
print(f"{year2} is a leap year: {is_leap_year(year2)}")
print(f"{year3} is a leap year: {is_leap_year(year3)}")
print(f"{year4} is a leap year: {is_leap_year(year4)}")
```

```
2024 is a leap year: True
1900 is a leap year: False
2000 is a leap year: True
2023 is a leap year: False
```

OBSERVATION:

The code effectively defines a function is_leap_year that correctly implements the rules for determining if a year is a leap year. It handles the conditions for divisibility by 4, 100, and 400 as required. The example usage with years 2024, 1900, 2000, and 2023 demonstrates the function's accuracy in identifying leap years according to the Gregorian calendar rules.

Note: Report should be submitted a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots

Evaluation Criteria:

Criteria	Max Marks
Two sorting implementations: Bubble sort (manual logic) and Built-in sort() (Task#1)	0.5
Counts vowels, consonants, and digits(Task#2)	0.5
Functional code that creates a .txt file, writes content to it, and reads it back- Use cursor (Task#3)	0.5
Complete calculator code with user input and operation selection. (Task#4)	0.5
A functional program to check leap year with sample input/output-use Cursor (Task#5)	0.5
Total	**2.5 Marks**